

Chap.5 합성곱 신경망 I (Part 1 - 5.1, 5.2)

5.1 합성곱 신경망

5.1.1 합성곱층의 필요성

5.1.2 합성곱 신경망(Convolutional Neural Network, CNN) 구조

입력층 (Input layer)

합성곱층 (Convolutional layer)

풀링층 (Pooling layer)

완전연결층 (Fully connected layer)

출력층

5.1.3 1D, 2D, 3D 합성곱

1D 합성곱

2D 합성곱

3D 합성곱

3D 입력을 갖는 2D 합성곱

1 × 1 합성곱

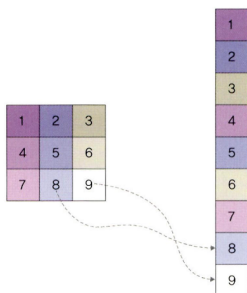
5.2 합성곱 신경망 맛보기

5.1 합성곱 신경망

5.1.1 합성곱층의 필요성

합성곱 신경망은 이미지나 영상을 처리하는 데 유용함.

e.g. 이미지 분석

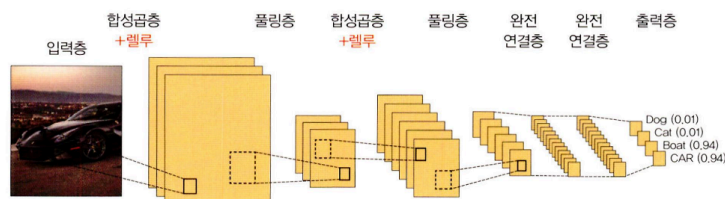


- 이미지 분석은 그림과 같은 3 x 3 배열을 오른쪽과 같이 펼쳐서(flattening) 각 픽셀에 가중치를 곱해 은닉층으로 전달
 - 문제점 : 이미지를 펼쳐서 분석하면 데이터의 공간적 구조를 무시하게 됨
- 방지하기 위해 '합성곱층' 도입

5.1.2 합성곱 신경망(Convolutional Neural Network, CNN) 구조

합성곱 신경망 : 음성 인식이나 이미지/영상 인식에서 주로 사용되는 신경망.

다차원 배열 데이터를 처리하도록 구성되어, 컬러 이미지 같은 다차원 배열 처리에 특화되어 있음.



계층 다섯 개로 구성됨.

1. 입력층
2. 합성곱층
3. 풀링층

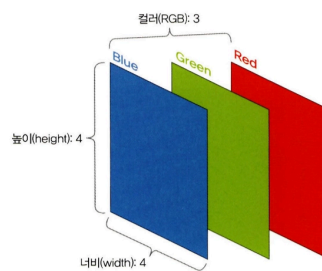
4. 완전연결층

5. 출력층

합성곱 신경망은 convolutional layer와 pooling layer를 거치면서 입력 이미지의 주요 특성 벡터(feature vector)를 추출. 그 후 추출된 주요 특성 벡터들은 fully-connected layer를 거치며 1차원 벡터로 변환되며, 마지막으로 출력층에서 활성화 함수인 softmax 함수를 사용하여 최종 결과가 출력됨.

입력층 (Input layer)

- 입력 이미지 데이터가 최초로 거치게 되는 계층
- 이미지는 단순 1차원의 데이터가 아닌 높이(height), 너비(width), 채널(channel)의 값을 가지는 3차원 데이터
 - 채널 : 이미지가 gray scale → 1, 컬러(RGB) → 3
 - e.g. 이미지 shape : (4, 4, 3)



합성곱층 (Convolutional layer)

- 입력 데이터에서 특성을 추출하는 역할 수행
 - 특성 추출 방법
 - 입력 이미지가 들어왔을 때, 이미지에 대한 특성을 감지하기 위해 커널이나 필터 사용
 - 커널/필터는 이미지의 모든 영역을 훑으면서 특성을 추출함. → 추출된 결과물 : feature map
 - 이때 커널은 3 x 3, 5 x 5 크기로 적용되는 것이 일반적
 - Stride라는 지정된 간격에 따라 순차적으로 이동함
- ▼ Stride = 1일 때 이동 과정 — 이미지 크기 : (6, 6, 1), 3x3 크기의 커널/필터, Gray scale

1. 입력 이미지에 3x3 필터 적용

입력 이미지와 필터를 포개 놓고 대응되는 숫자끼리 곱한 후 모두 더한다.



$$(1 \times 1) + (0 \times 0) + (0 \times 1) + (0 \times 0) + (1 \times 1) + (0 \times 0) + (0 \times 1) + (0 \times 0) + (1 \times 1) = 3$$

2. 필터가 1만큼 이동 (stride=1)



$$(0 \times 1) + (0 \times 0) + (0 \times 1) + (1 \times 0) + (0 \times 1) + (0 \times 0) + (0 \times 1) + (1 \times 0) + (1 \times 1) = 1$$

3. 필터가 1만큼 두 번째 이동



4. 필터가 1만큼 세 번째 이동



5. 필터가 1만큼 네 번째 이동



...

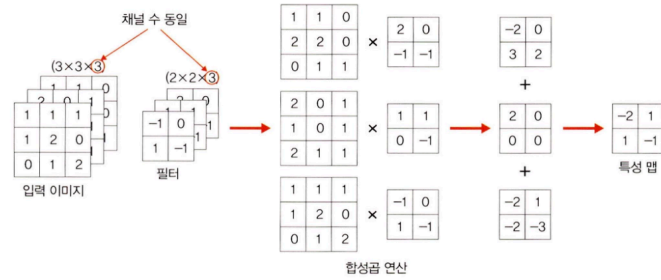
16. 필터가 1만큼 마지막으로 이동



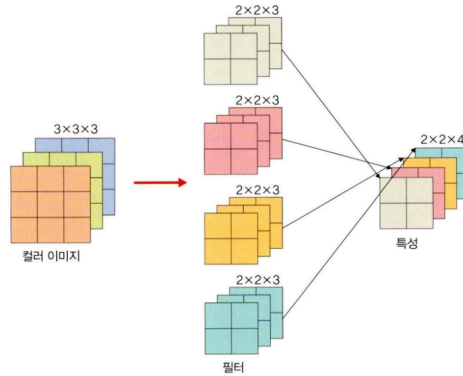
→ 커널과 스트라이드의 상호작용으로 원본 (6, 6, 1) 크기가 (4, 4, 1) 크기의 특성 맵으로 줄어듦.

▼ 컬러 이미지의 합성곱

- 필터 채널이 3 (필터 개수는 한 개)
- RGB 각각에 서로 다른 가중치로 합성곱을 적용한 후 결과를 더해줌
- 그 외 스트라이드 및 연산하는 방법은 동일



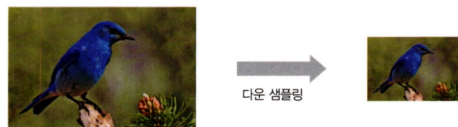
- 필터가 두 개 이상인 합성곱의 경우, 필터 각각은 특성 추출 결과의 채널이 됨.



- 합성곱층의 구조
 - 입력 데이터 : $W_1 \times H_1 \times D_1$ (W_1 : 가로, H_1 : 세로, D_1 : 채널 또는 깊이)
 - 하이퍼파라미터
 - 필터 개수 : K
 - 필터 크기 : F
 - 스트라이드 : S
 - 패딩 : P
 - 출력 데이터
 - $W_2 = (W_1 - F + 2P)/S + 1$
 - $H_2 = (H_1 - F + 2P)/S + 1$
 - $D_2 = K$

풀링층 (Pooling layer)

- 합성곱층과 유사하게 특성 맵의 차원을 다운 샘플링하여 연산량을 감소시키고, 주요한 특성 벡터를 추출하여 학습을 효과적으로 할 수 있게 함
 - 다운 샘플링 (sub-sampling)? 다음 그림과 같이 이미지를 축소하는 것



- 풀링 연산에는 두 가지가 사용됨
 - 최대 풀링 (max pooling) : 대상 영역에서 최댓값을 추출 ← CNN에서 주로 사용

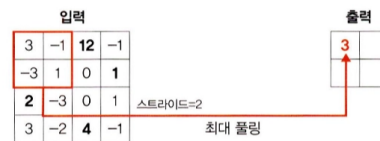
- 평균 풀링 (average pooling) 대상 영역에서 평균을 반환

CNN에서 max pooling을 많이 사용하는 이유? average pooling은 각 커널 값을 평균화 시켜 중요한 가중치를 갖는 값의 특성이 희미해질 수 있기 때문

▼ Max Pooling의 연산 과정

1. 첫 번째 max pooling 과정

3, -1, -3, 1 값 중 최댓값(3) 선택



2. 두 번째

12, -1, 0, 1 값 중 최댓값 선택



3. 세 번째

2, -3, 3, -2 값 중 최댓값 선택



4. 네 번째

0, 1, 4, -1 값 중 최댓값 선택



▼ Average Pooling 계산 과정

Max pooling과 유사한 방식으로 진행 but, 각 필터의 평균으로 계산

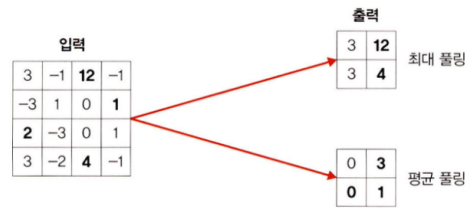
$$0 = (3 + (-1) + (-3) + 1) / 4$$

$$3 = (12 + (-1) + 0 + 1) / 4$$

$$0 = (2 + (-3) + 3 + (-2)) / 4$$

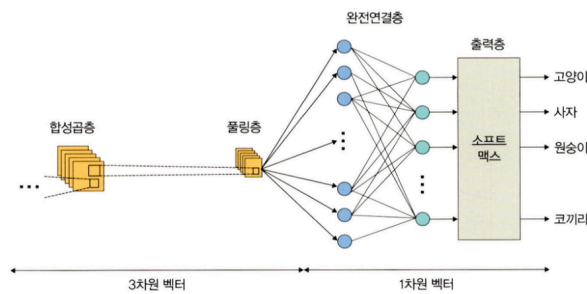
$$1 = (0 + 1 + 4 + (-1)) / 4$$

- Max pooling과 Average pooling 결과 비교



- Pooling 층의 구조
 - 입력 데이터 : $W_1 \times H_1 \times D_1$
 - 하이퍼파라미터
 - 필터 크기 : F
 - 스트라이드 : S
 - 출력 데이터
 - $W_2 = (W_1 - F) / S + 1$
 - $H_2 = (H_1 - F) / S + 1$
 - $D_2 = D_1$

완전연결층 (Fully connected layer)



합성곱층과 풀링층을 거치면서 차원이 축소된 feature map은 최종적으로 완전연결층(Fully connected layer)으로 전달됨. 이 과정에서 이미지는 3차원 벡터에서 1차원 벡터로 펼쳐지게(flatten) 됨.

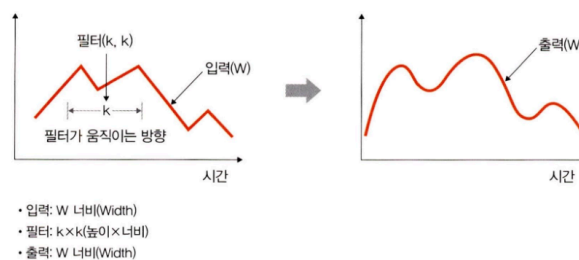
출력층

출력층(Output layer)에서는 소프트맥스 활성화 함수가 사용되는데, 입력받은 값을 0~1 사이의 값으로 출력함. 따라서 마지막 출력층의 소프트맥스 함수를 사용하여 이미지가 각 레이블에 속할 확률 값이 출력되며, 이때 가장 높은 확률 값을 갖는 레이블이 최종 값으로 선정됨.

5.1.3 1D, 2D, 3D 합성곱

합성곱은 이동하는 방향의 수와 출력 형태에 따라 1D, 2D, 3D로 분류할 수 있다.

1D 합성곱

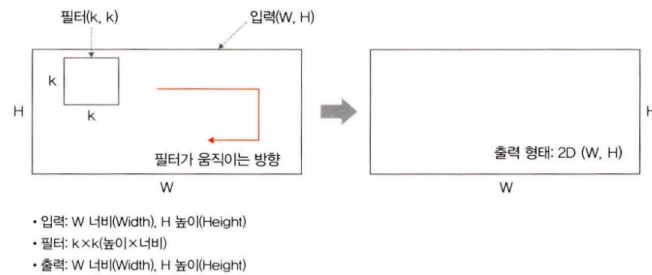


필터가 시간을 축으로 좌우로만 이동할 수 있는 합성곱.

입력(W)과 필터(k)에 대한 출력은 W가 됨.

e.g. 입력이 [1, 1, 1, 1, 1], 필터가 [0.25, 0.5, 0.25] 인 경우, 출력은 [1, 1, 1]

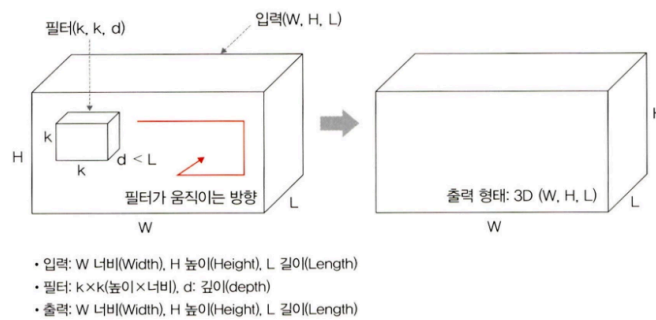
2D 합성곱



필터가 그림과 같이 방향 두 개로 움직이는 형태.

입력(W, H)과 필터(k, k)에 대한 출력은 (W, H)가 되며, 출력 형태는 2D 행렬이 됨.

3D 합성곱

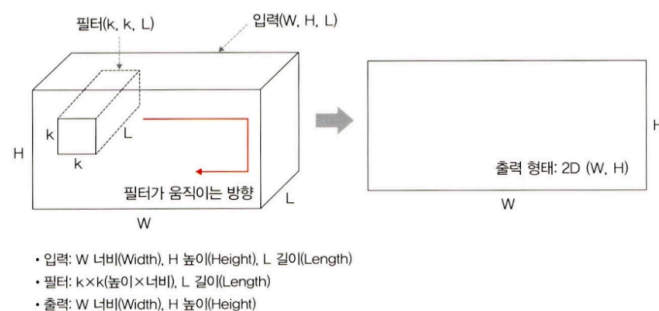


필터가 움직이는 방향이 그림과 같이 세 개.

입력(W, H, L)에 대해 필터(k, k, d)를 적용하면 출력으로 (W, H, L)을 갖는 형태가 3D 합성곱.

출력은 3D 형태. 이때 $d < L$ 을 유지하는 것이 중요.

3D 입력을 갖는 2D 합성곱



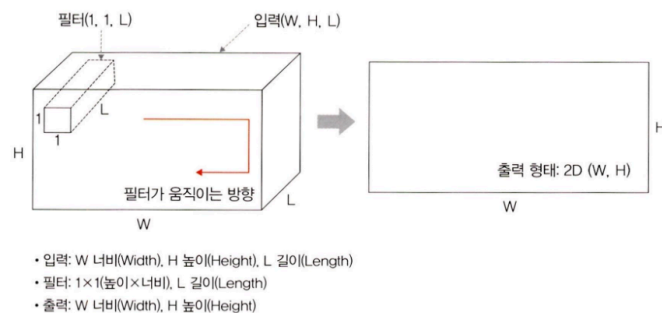
입력이 (224x224x3, 112x112x32)와 같은 3D 형태임에도 출력 형태가 3D가 아닌 2D 행렬을 취하는 것이 '3D 입력을 갖는 2D 합성곱' 이다.

필터에 대한 길이(L)가 입력 채널의 길이(L)와 같아야 하기 때문에 이와 같은 합성곱 형태가 만들어짐.

입력(W, H, L)에 필터(k, k, L)를 적용하면 출력은 (W, H)가 됨. 이때 필터는 그림과 같이 두 방향으로 움직이며 출력 형태는 2D 행렬이 됨.

3D 입력을 갖는 2D 합성곱의 대표적 사례로는 LeNet-5와 VGG가 있으며, 이들에 대해서는 6장에서 자세히 다룰 예정.

1×1 합성곱



1×1 합성곱은 3D 형태로 입력됨.

입력(W, H, L)에 필터(1, 1, L)를 적용하면 출력은 (W, H)가 됨.

1×1 합성곱에서 채널 수를 조정해서 연산량이 감소되는 효과가 있으며, 대표적 사례로는 GoogLeNet이 있음. 이 모델 역시 6장에서 자세히 다룰 예정

5.2 합성곱 신경망 맛보기

fashion_mnist 데이터셋을 사용해 합성곱 신경망을 직접 구현하기

▼ fashion_mnist 데이터셋

torchvision에 내장된 예제 데이터. 운동화, 셔츠, 샌들 같은 작은 이미지의 모음으로, 기본 MNIST 데이터셋처럼 열 가지로 분류될 수 있는 28x28 픽셀의 이미지 7만개로 구성됨.

train_images : 0에서 255 사이의 값을 갖는 28x28 크기의 넘파이 배열

train_labels : 0에서 9까지 정수 값을 갖는 배열. 정수 이미지(운동화, 셔츠 등)의 클래스를 나타내는 레이블.

```
0 : T-Shirt
1 : Trouser
2 : Pullover
3 : Dress
4 : Coat
5 : Sandal
6 : Shirt
7 : Sneaker
8 : Bag
9 : Ankle Boot
```

```
# 라이브러리 호출
import numpy as np
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
from torch.autograd import Variable
import torch.nn.functional as F

import torchvision
import torchvision.transforms as transforms # 데이터 전처리를 위해 사용하는 라이브러리
from torch.utils.data import Dataset, DataLoader
```



```
# CPU / GPU 장치 확인
device = torch.device("mps" if torch.backends.mps.is_available() else "cpu") # 맥북이기 때문에 mps 사용
print(device)
```

```
# fashion_mnist 데이터셋 내려받기
train_dataset = torchvision.datasets.FashionMNIST("data/", download=True, transform=transforms.Compose([transforms.ToTensor()]))
test_dataset = torchvision.datasets.FashionMNIST("data/", download=True, train=False, transform=transforms.Compose([transforms.ToTensor()]))
```

- `torchvision.datasets` 는 `torch.utils.data.Dataset` 의 하위클래스로 다양한 데이터셋(CIFAR, COCO, MNIST, ImageNet 등)을 포함한다.
- `torchvision.datasets` 에서 사용하는 주요한 파라미터

```
torchvision.datasets.FashionMNIST("../chap05/data", download=True, transform=transforms.Compose([transforms.ToTensor()]))
```

- `"../chap05/data"` : FashionMNIST를 내려받을 위치를 정함
 - `download=True` : 첫 번째 파라미터의 위치에 해당 데이터셋이 있는지 확인 후 내려받음
 - `transform=transforms.Compose([transforms.ToTensor()])` : 이미지를 텐서(0~1)로 변경함
- 데이터 다운로드 결과

```
Downloading http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/train-images-idx3-ubyte.gz
Downloading http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/train-images-idx3-ubyte.gz
100.0%
Extracting data/FashionMNIST/raw/train-images-idx3-ubyte.gz to data/FashionMNIST/raw

Downloading http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/train-labels-idx1-ubyte.gz
Downloading http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/train-labels-idx1-ubyte.gz
100.0%
Extracting data/FashionMNIST/raw/train-labels-idx1-ubyte.gz to data/FashionMNIST/raw

Downloading http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/t10k-images-idx3-ubyte.gz
Downloading http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/t10k-images-idx3-ubyte.gz
100.0%
Extracting data/FashionMNIST/raw/t10k-images-idx3-ubyte.gz to data/FashionMNIST/raw

Downloading http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/t10k-labels-idx1-ubyte.gz
Downloading http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/t10k-labels-idx1-ubyte.gz
100.0%
Extracting data/FashionMNIST/raw/t10k-labels-idx1-ubyte.gz to data/FashionMNIST/raw
```

```
# fashion_mnist 데이터를 데이터로더에 전달
train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=100)
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=100)
```

- `torch.utils.data.DataLoader()` 를 사용하여 원하는 크기의 배치 단위로 데이터를 불러오거나, 순서가 무작위로 섞이도록 (shuffle) 할 수 있음
- 데이터로더에서 사용하는 파라미터

```
torch.utils.data.DataLoader(train_dataset, batch_size=100)
```

- `train_dataset` : 데이터를 불러올 데이터셋을 지정
- `batch_size` : 데이터를 배치로 묶어줌. 여기에서는 `batch_size=100`으로 지정했기 때문에 100개 단위로 데이터를 묶어서 불러옴.

```
# 분류에 사용될 클래스 정의
labels_map = {0:'T-Shirst', 1:'Trouser', 2:'Pullover', 3:'Dress', 4:'Coat', 5:'Sandal', 6:'Shirt', 7:'Sneaker', 8:'Bag', 9:'Ankle Boot'} # 열 개의 클래스

fig = plt.figure(figsize=(8,8)); # 출력할 이미지의 가로세로 길이로 단위는 inch
columns = 4
rows = 5
for i in range(1, columns*rows + 1):
    img_xy = np.random.randint(len(train_dataset)) # 설명(1)
    img = train_dataset[img_xy][0][0,:,:] # 설명(2)
    fig.add_subplot(rows, columns, i)
    plt.title(labels_map[train_dataset[img_xy][1]])
    plt.axis('off')
    plt.imshow(img, cmap='gray')
plt.show() # 20개의 이미지 데이터를 시각적으로 표현
```

• 설명 (1)

`np.random` 은 무작위로 데이터를 생성할 때 사용. 또한, `np.random.randint()` 는 이산형 분포를 갖는 데이터에서 무작위 표본을 추출할 때 사용.

- `np.random.randint(len(train_dataset))` : 0~(train_dataset의 길이) 값을 갖는 분포에서 랜덤한 숫자 한 개를 생성하라는 의미

▼ c.f. `random.randint` 와 유사하게 사용되는 `random.rand` 와 `random.randn` 의 예시

```
> import numpy as np
> np.random.randint(10) ----- 0~10의 임의의 숫자를 출력
2

> np.random.randint(1, 10) ----- 1~9의 임의의 숫자를 출력
1

> np.random.rand(8) ----- 0~1 사이의 정규표준분포 난수를 행렬로 (1×8) 출력

array([0.89213233, 0.24661652, 0.73743451, 0.25592822, 0.49036819,
       0.97493688, 0.6356506 , 0.50091059])

> np.random.rand(4, 2) ----- 0~1 사이의 정규표준분포 난수를 행렬로 (4×2) 출력
array([[0.11760522, 0.08913278],
       [0.78621819, 0.1720912 ],
       [0.46788007, 0.30779366],
       [0.66442201, 0.40509264]])

> np.random.randn(8) ----- 평균이 0이고, 표준편차가 1인 가우시안 정규분포 난수를 행렬로 (1×8) 출력
array([ 0.25879494, -0.43633211,  0.61310559,  0.89605309, -0.8632414,
       -0.67915832, -0.49204324, -0.43216491])

> np.random.randn(4, 2) ----- 평균이 0이고, 표준편차가 1인 가우시안 정규분포 난수를 행렬로 (4×2) 출력
array([[ -0.51816619, -0.91865557],
       [-0.33232116,  1.21916794],
       [-1.09654634, -0.42127499],
       [ 1.40003186, -1.34076957]])
```

• 설명 (2)

`train_dataset`을 이용한 3차원 배열을 생성.

▼ e.g. 배열에 대한 사용

```

> import numpy as np
> examp = np.arange(0, 100, 3) ----- 1~99의 숫자에서 3씩 건너편 행렬을 생성
> examp.resize(6, 4) ----- 행렬의 크기를 6×4로 조정
> examp
array([[ 0,  3,  6,  9],
       [12, 15, 18, 21],
       [24, 27, 30, 33],
       [36, 39, 42, 45],
       [48, 51, 54, 57],
       [60, 63, 66, 69]])

> examp[3] ----- 3행에 해당하는 모든 요소(값)들을 출력(행과 열은 0부터 시작)
array([36, 39, 42, 45])

> examp[3, 3] ----- 3행의 3번째 열에 대한 값(요소)을 출력
45

> examp[3][3] ----- 3행의 3번째 열에 대한 값(요소)을 출력하기 때문에 바로 앞의 결과와 동일
45

```

- `train_dataset[img_xy][0][0,:,:]` 의미는 다음 예시로 출력 결과 유추 가능

```

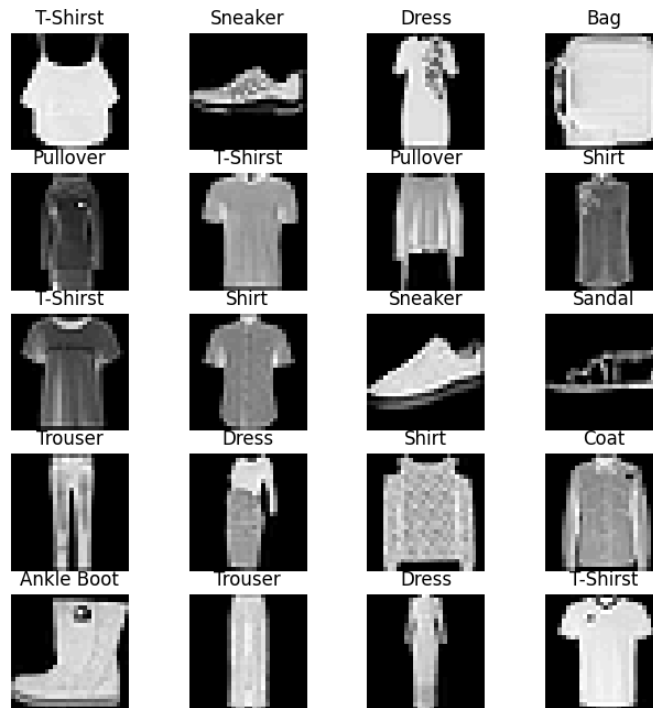
> examp = np.arange(0, 500, 3)
> examp.resize(3, 5, 5)
> examp
array([[[ 0,  3,  6,  9, 12],
        [15, 18, 21, 24, 27],
        [30, 33, 36, 39, 42],
        [45, 48, 51, 54, 57],
        [60, 63, 66, 69, 72]],
       [[ 75, 78, 81, 84, 87],
        [90, 93, 96, 99, 102],
        [105, 108, 111, 114, 117],
        [120, 123, 126, 129, 132],
        [135, 138, 141, 144, 147]],
       [[150, 153, 156, 159, 162],
        [165, 168, 171, 174, 177],
        [180, 183, 186, 189, 192],
        [195, 198, 201, 204, 207],
        [210, 213, 216, 219, 222]]])

> examp[2][0][3]
159

```

`examp[2][0][3]` 과 같이 `train_dataset[img_xy][0][0,:,:]` 의미는 train_dataset에서 [img_xy][0][0,:,:]에 해당하는 요소 값을 가져오겠다는 의미.

▼ 코드 실행 결과



합성곱 신경망과 합성곱 신경망이 아닌 심층 신경망의 비교를 위해, 먼저 심층 신경망을 생성한 후 학습시키기. (ConvNet이 적용되지 않은 네트워크 생성)

```
# 심층 신경망 모델 생성
class FashionDNN(nn.Module):
    def __init__(self): # 설명(1)
        super(FashionDNN, self).__init__()
        self.fc1 = nn.Linear(in_features=784, out_features=256) # 설명(2)
        self.drop = nn.Dropout(0.25) # 설명(3)
        self.fc2 = nn.Linear(in_features=256, out_features=128)
        self.fc3 = nn.Linear(in_features=128, out_features=10)

    def forward(self, input_data): # 설명(4)
        out = input_data.view(-1, 784) # 설명(5)
        out = F.relu(self.fc1(out)) # 설명(6)
        out = self.drop(out)
        out = F.relu(self.fc2(out))
        out = self.fc3(out)
        return out
```

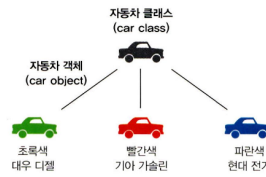
- 설명 (1)

```
def __init__(self):
```

클래스(class) 형태의 모델은 항상 `torch.nn.Module` 을 상속받음. `__init__()` 은 객체가 갖는 속성 값을 초기화하는 역할을 하며, 객체가 생성될 때 자동으로 호출됨.

`super(FashionDNN, self).__init__()` 은 FashionDNN이라는 부모(super) 클래스를 상속받겠다는 의미.

- ▼ 객체



객체(object)란 메모리를 할당받아 프로그램에서 사용되는 모든 데이터를 의미. 변수, 함수 등은 모두 객체.

객체명 = 클래스명() 과 같은 방식으로 사용

▼ 클래스와 함수

- 함수(function)란?

하나의 특정 작업을 수행하기 위해 독립적으로 설계된 프로그램 코드.

함수의 호출은 특정 작업만 수행할 뿐 그 결과값을 계속 사용하기 위해서는 반드시 어딘가에 따로 그 값을 저장해야함. 즉, 함수를 포함한 프로그램 코드의 일부를 재사용하기 위해서는 해당 함수뿐만 아니라 데이터가 저장되는 변수까지도 한꺼번에 관리해야 함.

- 클래스(class)란?

함수뿐만 아니라 관련된 변수까지도 한꺼번에 묶어서 관리하고 재사용할 수 있게 해주는 것.

- 클래스와 함수의 차이

```
# 함수 예시
def add(num1, num2): # 함수 정의(num1, num2를 받아서 더해 주는 함수)
    result = num1 + num2
    return result

print(add(1, 2))
print(add(2, 3))

''' 실행 결과
3
5
'''
```

```
# 클래스 예시
class Calc:
    def __init__(self): # 객체를 생성할 때 호출하면 실행되는 초기화 함수
        self.result = 0

    def add(self, num1, num2):
        self.result = num1 + num2
        return self.result

obj1 = Calc()
obj2 = Calc()

print(obj1.add(1, 2))
print(obj1.add(2, 3))
print('-----')
print(obj2.add(2, 2))
print(obj2.add(2, 3))

''' 실행 결과
3
```

```

5
-----
4
5
...

```

두 개의 객체는 독립적으로 연산됨. 개별적 함수로 구현했다면 복잡했을 코드가 클래스 사용으로 간결해짐.

- 설명 (2)

nn은 딥러닝 모델(네트워크) 구성에 필요한 모듈이 모여 있는 패키지이며, Linear는 단순 선형 회귀 모델을 만들 때 사용. 이때 사용되는 파라미터 :

```
nn.Linear(in_features=784, out_features=256)
```

- `in_features` : 입력의 크기(input size)
- `out_features` : 출력의 크기(output size)

실제로 데이터 연산이 진행되는 `forward()` 부분에는 첫 번째 파라미터 값만 넘겨주게 되며, 두 번째 파라미터에서 정의된 크기가 `forward()` 연산의 결과가 됨.

- 설명 (3)

`torch.nn.Dropout(p)` 는 p만큼의 비율로 텐서의 값이 0이 되고, 0이 되지 않는 값들은 기존 값에 $(1/(1-p))$ 만큼 곱해져 커짐.

e.g. $p=0.3$ 일 때, 전체값 중 0.3의 확률로 0이 된다는 것이며, 0이 되지 않는 0.7에 해당하는 값은 $(1/(1-0.7))$ 만큼 커짐

- 설명 (4)

`forward()` 함수는 모델이 학습 데이터를 입력받아 forward propagation을 진행시키며, 반드시 forward라는 이름의 함수여야 함.

즉, `forward()` 는 모델이 학습 데이터를 입력받아서 forward propagation 연산을 진행하는 함수이며, 객체를 데이터와 함께 호출하면 자동으로 실행됨.

c.f. Forward propagation 연산이란? $H(x)$ 식에 입력 x 로부터 예측된 y 를 얻는 것.

* $H(x) = \text{sigmoid}(x_1w_1 + x_2w_2 + b)$ 처럼 표현된 수식. 이때 $H(x)$ 는 각 모델에서 사용하는 활성화 함수 및 입력 값에 따라 다름.

- 설명 (5)

파이토치에서 사용하는 view는 넘파이의 reshape과 같은 역할로 텐서의 shape을 변경해줌.

- 설명 (6)

활성화 함수를 지정할 때는 다음 두 가지 방법이 가능

- `F.relu()` : `forward()` 함수에서 정의
- `nn.ReLU()` : `__init__()` 함수에서 정의

둘의 차이는 간단히 '사용하는 위치'.

근본적으로는 `nn.functional.xx()` (혹은 `F.xx()`)와 `nn.xx()` 는 사용 방법에 차이 있음.

▼ nn 사용 코드

```

import torch
import torch.nn as nn

inputs = torch.randn(64, 3, 244, 244)
conv = nn.Conv2d(in_channels=3, out_channels=64, kernel_size=3, padding=1) # 세
개의 채널이 입력되어 64개의 채널이 출력되기 위한 연산으로 3x3 크기의 커널 사용
outputs = conv(inputs)
layer = nn.Conv2d(1, 1, 3)

```

▼ nn.functional 사용 코드

```
import torch.nn.functional as F

inputs = torch.randn(64, 3, 244, 244)
weight = torch.randn(64, 3, 3, 3)
bias = torch.randn(64)
outputs = F.conv2d(inputs, weight, bias, padding=1)
```

`nn.Conv2d` 에서 input_channel과 output_channel을 사용해서 연산했다면 `functional.conv2d` 는 입력(input)과 가중치(weight) 자체를 직접 넣어줌 (가중치를 전달해야 할 때마다 가중치 값을 새로 정의해야 함). 그 외에 채워야 하는 파라미터들은 `nn.Conv2d` 와 비슷함.

- `nn.xx` 와 `nn.functional.xx` 사용 방법 비교

구분	nn.xx	nn.functional.xx
형태	nn.Conv2d: 클래스 nn.Module 클래스를 상속받아 사용	nn.functional.conv2d: 함수 def function (input)으로 정의된 순수한 함수
호출 방법	먼저 하이퍼파라미터를 전달한 후 함수 호출을 통해 데이터 전달	함수를 호출할 때 하이퍼파라미터, 데이터 전달
위치	nn.Sequential 내에 위치	nn.Sequential에 위치할 수 없음
파라미터	파라미터를 새로 정의할 필요 없음	가중치를 수동으로 전달해야 할 때마다 자체 가중치를 정의

모델 학습 전에 손실 함수, 학습률(learning rate), 옵티마이저(optimizer)에 대해 정의.

```
# 심층 신경망에서 필요한 파라미터 정의
learning_rate = 0.001
model = FashionDNN()
model.to(device)

criterion = nn.CrossEntropyLoss() # 분류 문제에서 사용하는 손실 함수
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate) # 설명
print(model)
```

- 옵티마이저를 위한 경사 하강법은 Adam을 사용하며, 학습률을 의미하는 lr은 0.001을 사용한다는 의미.

▼ 코드 실행 결과

```
FashionDNN(
  (fc1): Linear(in_features=784, out_features=256, bias=True)
  (drop): Dropout(p=0.25, inplace=False)
  (fc2): Linear(in_features=256, out_features=128, bias=True)
  (fc3): Linear(in_features=128, out_features=10, bias=True)
)
```

심층 신경망에 데이터를 적용해 모델 학습시키기

```
# 심층 신경망을 이용한 모델 학습
num_epochs = 5
count = 0
loss_list = [] # 설명(1)
iteration_list = []
accuracy_list = []
```

```

predictions_list = []
labels_list = []

for epoch in range(num_epochs):
    for images, labels in train_loader: # 설명(2)
        images, labels = images.to(device), labels.to(device) # 설명(3)

        train = Variable(images.view(100, 1, 28, 28)) # 설명(4)
        labels = Variable(labels)

        outputs = model(train) # 학습 데이터를 모델에 적용
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        count += 1

    if not (count % 50): # count를 50으로 나눴을 때 나머지가 0이 아니면 실행
        total = 0
        correct = 0
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            labels_list.append(labels)
            test = Variable(images.view(100, 1, 28, 28))
            outputs = model(test)
            predictions = torch.max(outputs, 1)[1].to(device)
            predictions_list.append(predictions)
            correct += (predictions == labels).sum()
            total += len(labels)

        accuracy = correct * 100 / total # 설명(5)
        loss_list.append(loss.data) # 설명 (1')
        iteration_list.append(count)
        accuracy_list.append(accuracy)

    if not (count % 500):
        print("Iteration: {}, Loss: {}, Accuracy: {}%".format(count, loss.data, ac
curacy))

```

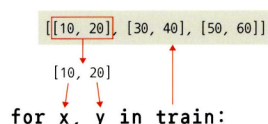
- 설명 (1), (1')

일반적으로 배열이나 행렬과 같은 리스트(list)를 사용하는 방법은 다음과 같다.

- (1)과 같이 비어 있는 배열이나 행렬 만들기
- (1')처럼 append 메서드를 이용해 데이터를 하나씩 추가

- 설명 (2)

for 구문을 사용해 레코드(행, 가로줄)를 하나씩 가져옴. 이때 `for x, y in train:` 과 같이 in 앞에 변수를 두 개 지정해 주면 레코드에서 요소 두 개를 꺼내 오겠다는 의미.



- 설명 (3)

모델이 데이터를 처리하기 위해서는 모델과 데이터가 동일한 장치(CPU/GPU)에 있어야 함. 앞에서 사용한 device와 같은 device 사용.

- 설명 (4)

Autograd는 자동 미분을 수행하는 파이토치의 핵심 패키지로, 자동 미분에 대한 값을 저장하기 위해 테이프(tape) 사용.

forward 단계에서 테이프는 수행하는 모든 연산을 저장.

backward 단계에서 저장된 값들을 꺼내서 사용.

→ Autograd는 Variable을 사용해 역전파를 위한 미분 값을 자동으로 계산해줌. 따라서 자동 미분을 계산하기 위해서는 torch.autograd 패키지 안에 있는 Variable을 이용해야 동작함.

- 설명 (5)

분류 문제에 대한 정확도는 (정확한 예측)/(전체 예측) 의 비율로 표현.

```
# 정확도 코드
classification accuracy = correct predictions / total predictions

# 백분율로 표시
classification accuracy = correct predictions / total predictions * 100

# 값을 반전시켜 오분류율 또는 오류율로 표현
error rate = (1 - (correct predictions / total predictions)) * 100
```

분류 문제에서 클래스가 세 개 이상일 때 주의해야 할 점

- 정확도가 80% 이상이었다고 할 때, 80%라는 값이 모든 클래스가 동등하게 고려된 것인지, 특정 클래스의 분류가 높았던 것인지에 대해 알 수 없음에 유의해야 함.
- 정확도가 90% 이상이었다고 할 때, 100개의 데이터 중 90개가 하나의 클래스에 속할 경우, 90%의 정확도는 높다고 할 수 없음. 즉, 모든 데이터를 특정 클래스에 속한다고 예측해도 90%의 예측 결과가 나오기 때문에 데이터 특성에 따라 정확도를 잘 관측해야 함.

▼ 코드 실행 결과

```
Iteration: 500, Loss: 0.5822047591209412, Accuracy: 83.02999877929688%
Iteration: 1000, Loss: 0.5423489809036255, Accuracy: 84.1500015258789%
Iteration: 1500, Loss: 0.32638972997665405, Accuracy: 85.0199966430664%
Iteration: 2000, Loss: 0.30313289165496826, Accuracy: 85.23999786376953%
Iteration: 2500, Loss: 0.25221240520477295, Accuracy: 86.18000030517578%
Iteration: 3000, Loss: 0.29167798161506653, Accuracy: 86.79000091552734%
```

높은 정확도 수치를 보여줌.

합성곱 신경망 생성하기

```
# 합성곱 네트워크 생성
class FashionCNN(nn.Module):
    def __init__(self):
        super(FashionCNN, self).__init__()
        self.layer1 = nn.Sequential( # 설명(1)
            nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1), # 설명(2)
            nn.BatchNorm2d(32), # 설명(3)
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2) # 설명(4)
        )

        self.layer2 = nn.Sequential(
```

```

        nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3),
        nn.BatchNorm2d(64),
        nn.ReLU(),
        nn.MaxPool2d(2)
    )
    self.fc1 = nn.Linear(in_features=64*6*6, out_features=600) # 설명(5)
    self.drop = nn.Dropout2d(0.25)
    self.fc2 = nn.Linear(in_features=600, out_features=120)
    self.fc3 = nn.Linear(in_features=120, out_features=10) # 마지막 계층의 out_featu
res는 클래스 개수

    def forward(self, x):
        out = self.layer1(x)
        out = self.layer2(out)
        out = out.view(out.size(0), -1) # 설명(6)
        out = self.fc1(out)
        out = self.drop(out)
        out = self.fc2(out)
        out = self.fc3(out)
        return out

```

- 설명 (1)

`nn.Sequential` 을 사용하면 `__init__()` 에서 사용할 네트워크 모델들을 정의해 줄 뿐만 아니라, `forward()` 함수에서 구현될 순전파를 계층(layer) 형태로 좀 더 가독성이 뛰어난 코드로 작성할 수 있음.

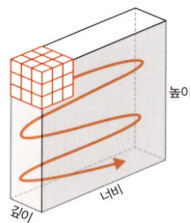
즉, `nn.Sequential` 은 계층을 차례로 쌓을 수 있도록 $Wx + b$ 와 같은 수식과 활성화 함수를 연결해 주는 역할. 여러 개의 계층을 하나의 컨테이너에 구현하는 방법.

- 설명 (2)

합성곱층(conv layer)은 합성곱 연산을 통해서 이미지의 특징을 추출함. 합성곱이란 커널(또는 필터)이라는 $n \times m$ 크기의 행렬이 높이(height) x 너비(width) 크기의 이미지를 처음부터 끝까지 훑으면서 각 원소 값끼리 곱한 후 모두 더한 값을 출력. 커널은 일반적으로 3×3 이나 5×5 를 사용하며 파라미터는 다음과 같다.

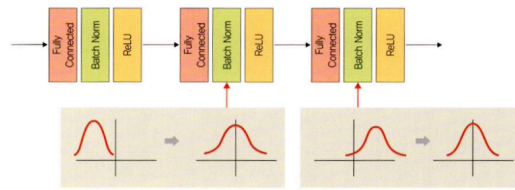
```
nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1)
```

- `in_channels` : 입력 채널 수. 흑백 이미지는 1, RGB 이미지는 3
c.f. 채널이란? 깊이.



- `out_channels` : 출력 채널의 수.
 - `kernel_size` : 커널 크기. 논문에 따라 필터라고도 함. 커널은 이미지 특징을 찾아내기 위한 공용 파라미터이며, CNN에서 학습 대상은 필터 파라미터가 됨. 입력 데이터를 스트라이드 간격으로 순회하며 합성곱을 계산.
 - `padding` : 패딩 크기. 출력 크기를 조정하기 위해 입력 데이터 주위에 0을 채움. 패딩 값이 클수록 출력 크기도 커짐.
- 설명 (3)

`BatchNorm2d`는 학습 과정에서 각 배치 단위별로 데이터가 다양한 분포를 가지더라도 평균과 분산을 이용해 정규화하는 것.



정규화를 통해 가우시안 형태로 만들면 평균 = 0, 표준편차 = 1로 데이터 분포가 조정됨.

• 설명 (4)

MaxPool2d는 이미지 크기를 축소시키는 용도로 사용. 풀링 계층은 합성곱층의 출력 데이터를 입력으로 받아서 출력 데이터 (activation map)의 크기를 줄이거나 특정 데이터를 강조하는 용도로 사용됨. 풀링 계층을 처리하는 방법은 1) max pooling, 2) average pooling, 3) min pooling 이 있으며, 이때 사용하는 파라미터는 다음과 같다.

```
nn.MaxPool2d(kernel_size=2, stride=2)
```

- **kernel_size** : m x n 행렬로 구성된 가중치
- **stride** : 입력 데이터에 커널을 적용할 때 이동할 간격. 스트라이드 값이 커지면 출력 크기는 작아짐.

• 설명 (5)

클래스 분류를 위해서 이미지 형태의 데이터를 배열 형태로 변환해 작업해야 함. 이때 Conv2d에서 사용하는 하이퍼파라미터 (패딩, 스트라이드) 값들에 따라 출력 크기(output size)가 달라짐. 이렇게 줄여든 출력 크기는 최종적으로 분류를 담당하는 fully connected layer 으로 전달됨.

```
nn.Linear(in_features=64*6*6, out_features=600)
```

- **in_features** : 입력 데이터의 크기를 의미. 이전까지 수행했던 Conv2d, MaxPool2d는 이미지 데이터를 입력으로 받아서 처리했지만, 그 출력 결과를 fully connected layer로 보내려면 1차원으로 변경해줘야 함. 공식은 아래 참고.

▼ Conv2d 계층에서의 출력 크기 구하는 공식

출력 크기 = $(W - F + 2P) / S + 1$

- W : 입력 데이터의 크기(input_volume_size)
- F : 커널 크기(kernel_size)
- P : 패딩 크기(padding_size)
- S : 스트라이드(strides)

e.g. 첫 번째 Conv2d 계층 출력 크기 계산

```
nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1)
```

```
''' 출력 크기
(784 - 3 + (2*1))/1 + 1 = 784
```

```
c.f. fashion_mnist의 입력 데이터 크기는 784이며, stride가 명시되어 있지 않다면 stride 기본값은 (1,1)
'''
```

→ 계산 결과 적용하면, 출력의 형태는 [32, 784, 784]

▼ MaxPool2d 계층에서의 출력 크기 구하는 공식

출력 크기 = IF / F

- IF : 입력 필터의 크기(input_filter_size, 또한 바로 앞의 Conv2d의 출력 크기이기도 함)
- F : 커널 크기(kernel_size)

e.g. 첫 번째 MaxPool2d 계층의 출력 크기

```
nn.MaxPool2d(kernel_size=2, stride=2)

''' 출력 크기
784 / 2 = 392 (784는 첫 번째 Conv2d에서 계산한 결과)
'''
```

→ 계산 결과를 적용하면, 출력의 형태는 [32, 392, 293]가 됨. 그리고 가장 앞의 32는 바로 앞 Conv2d 계층의 `out_channels`.

- `out_features` : 출력 데이터의 크기
- 설명 (6)
합성곱층에서 fully connected layer로 변경되기 때문에 데이터의 형태를 1차원으로 바꾸어 줌. 이때, `out.size(0)` 은 결국 100을 의미. 따라서 (100, ?) 크기의 텐서로 변경하겠다는 의미.

합성곱 네트워크를 사용하기 위한 파라미터 정의하기

```
# 합성곱 네트워크를 위한 파라미터 정의
learning_rate = 0.001
model = FashionCNN()
model.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
print(model)
```

▼ 코드 실행 결과

```
FashionCNN(
  (layer1): Sequential(
    (0): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU()
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (layer2): Sequential(
    (0): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1))
    (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU()
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (fc1): Linear(in_features=2304, out_features=600, bias=True)
  (drop): Dropout2d(p=0.25, inplace=False)
  (fc2): Linear(in_features=600, out_features=120, bias=True)
  (fc3): Linear(in_features=120, out_features=10, bias=True)
)
```

```
# 모델 학습 및 성능 평가
num_epochs = 5
count = 0
loss_list = []
iteration_list = []
accuracy_list = []

predictions_list = []
labels_list = []

for epoch in range(num_epochs):
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
```

```

train = Variable(images.view(100, 1, 28, 28))
labels = Variable(labels)

outputs = model(train)
loss = criterion(outputs, labels)
optimizer.zero_grad()
loss.backward()
optimizer.step()
count += 1

if not (count % 50):
    total = 0
    correct = 0
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        labels_list.append(labels)
        test = Variable(images.view(100, 1, 28, 28))
        outputs = model(test)
        predictions = torch.max(outputs, 1)[1].to(device)
        predictions_list.append(predictions)
        correct += (predictions == labels).sum()
        total += len(labels)

    accuracy = correct * 100 / total
    loss_list.append(loss.data)
    iteration_list.append(count)
    accuracy_list.append(accuracy)

if not (count % 500):
    print("Iteration: {}, Loss: {}, Accuracy: {}%".format(count, loss.data, ac
curacy))

```

▼ 코드 실행 결과

```

Iteration: 500, Loss: 0.4763776659965515, Accuracy: 87.9800033569336%
Iteration: 1000, Loss: 0.3360801339149475, Accuracy: 89.26000213623047%
Iteration: 1500, Loss: 0.2766103446483612, Accuracy: 88.0%
Iteration: 2000, Loss: 0.24637551605701447, Accuracy: 89.20999908447266%
Iteration: 2500, Loss: 0.18263313174247742, Accuracy: 89.58999633789062%
Iteration: 3000, Loss: 0.19264371693134308, Accuracy: 90.4800033569336%

```

심층 신경망과 비교하여 정확도가 약간 높음. 큰 차이는 없기 때문에, 좀 더 간편한 심층 신경망만 사용해도 무난할 것 같지만, 실제로 이미지 데이터가 많아지면 단순 심층 신경망으로는 정확한 특성 추출 및 분류가 불가능함.